# CSAPP 实验：Data Lab 实验报告

姓名：[任子蒙]　学号：[25303080004]

## 一、实验环境与截图

本实验在兼容的 Linux 环境下完成，使用 `make clean && make all` 重新编译后，分别执行了规则检查与正确性测试。最终取得了 **110/110** 的满分成绩。

### 1.1 规则检查截图

在此处展示执行 `./check_ops.py bits.c` 的结果，确认所有函数均符合操作数限制和合法运算符要求。

```
rzm@DESKTOP-DJ8DS86:~/DataLab-rzm$ ./test.sh
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 11/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 11/16 operators
roundEvenPow2: 12/24 operators
midpointTowardFirst: 19/32 operators
isBetweenEitherOrder: 29/48 operators
mul5Sat: 22/30 operators
classifyAdd3: 25/52 operators
floatScaleThreeHalves: 60/60 operators
floatRoundEven: 50/65 operators
float_i2f: 26/40 operators
bitCount: 29/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
operator check passed
```

图 1: 操作符规则检查通过截图

## 1.2 正确性测试截图

在此处展示执行 `./btest` 的最终结果，19 道题目的 Errors 均为 0，获得满分。

```
Score   Rating  Errors  Function
 1      1       0       signMask
 2      2       0       bitXor
 3      3       0       negativePart
 4      4       0       copyByteWithin
 4      4       0       logicalShift
 4      4       0       swapNibblePairs
 4      4       0       secondLowestZeroBit
 5      5       0       oddParity
 5      5       0       rotateRightBits
 5      5       0       roundEvenPow2
 5      5       0       midpointTowardFirst
 7      7       0       isBetweenEitherOrder
 7      7       0       mul5Sat
 7      7       0       classifyAdd3
 7      7       0       floatScaleThreeHalves
 10     10      0       floatRoundEven
 10     10      0       float_i2f
 10     10      0       bitCount
 10     10      0       bitReverse
Total points: 110/110
all checks passed: 110/110
```

图 2: 正确性测试满分截图

# 二、各函数实现思路

以下分别描述 P1 至 P19 的函数实现逻辑。在编写代码前，我先明确了输入输出范围、32 位补码模型以及 IEEE 754 单精度浮点数的位级结构。

## 整数部分 (P1 - P14)

**P1 signMask**

无参数，要求生成最高位为 1、其余位为 0 的掩码。只需将常数 1 左移 31 位，或利用算术右移特性，用 0 减去 1（在允许范围内使用位运算构造），再通过移位获得。仅需极少的操作数。

**P2 bitXor**

限制只能使用 `~` 和 `&`。利用离散数学中的德摩根定律：$x \oplus y = \sim(\sim(x\& \sim y)\& \sim(\sim x\& y))$。通过构造这两个非门与门组合，即可完美实现异或。

**P3 negativePart**

若 $x$ 为负数则返回 $x$，否则返回 0。只需将 $x$ 算术右移 31 位得到全 1（负数）或全 0（正数/零）的掩码，然后将原 $x$ 与该掩码进行按位与操作。

**P4 copyByteWithin**

将 $x$ 中的 `src` 字节复制到 `dst` 字节。思路是：先提取 `src` 字节，并将其移位到 `dst` 的位置；接着构造一个将 `dst` 位置 0 的掩码，清除原位置的旧数据；最后将两者进行按位或。移位量计算利用 $(src - dst) \ll 3$ 得到。

**P5 logicalShift**

实现逻辑右移。由于 C 语言中有符号 `>>` 是算术右移，因此在 $x \gg n$ 后，高位会被符号位填充。需要构造一个高 $n$ 位为 0，低 $32-n$ 位为 1 的掩码与其相与，清除符号扩展位。注意处理 $n = 0$ 的边界情况（通过 $(x \gg n)\&(\sim((1 \ll 31) \gg n) \ll 1)$ 避免移位量越界）。

**P6 swapNibblePairs**

交换相邻的半字节（Nibble）。分别用掩码 `0x0F0F0F0F` 和 `0xF0F0F0F0` 提取出奇数位和偶数位的半字节，前者左移 4 位，后者右移 4 位，最后进行按位或即可。

**P7 secondLowestZeroBit**

将最低位的第二个 0 变成 1，其余不变。先对 $x$ 取反得到 `~x`（此时 0 变为 1），提取第二低的 1：利用 `t = ~x & (~x - 1)` 消除最低位的 1，然后再与 `~x` 异或，得到仅包含第二低 0 的位置的掩码，最后与 $x$ 按位或。

**P8 oddParity**

计算 $x$ 中 1 的个数的奇偶性。采用分治法，先将高 16 位与低 16 位异或，再高 8 位与低 8 位异或，依此类推，最后只需检查最低位是否为 1 即可。

**P9 rotateRightBits**

循环右移 $n$ 位。考虑到实际位数按 32 取模（`n = n & 31`），先将 $x$ 右移 $n$ 位，同时把 $x$ 左移 $32-n$ 位（使用 `32 + ~n + 1` 构造），两者按位或。需注意 $n=0$ 时移位量要为 0 以免未定义行为。

**P10 roundEvenPow2**

除以 $2^n$ 并实现“四舍六入五成双”。将 $x$ 右移 $n$ 位得到商和余数。提取余数，比较其与 $2^{n-1}$ 的大小：大于则进 1；等于则根据商的最低位（奇偶）决定是否进 1。利用位移和加法构造比较掩码。

**P11 midpointTowardFirst**

求 $x$ 和 $y$ 的中点，向 $x$ 取整，且防溢出。核心公式为 $(x\&y)+((x\oplus y)\gg 1)$。如果两者异号且 $x<y$，或同号但需要向 $x$ 方向舍入时，可能需要额外加上修正项。

**P12 isBetweenEitherOrder**

判断 $x$ 是否在 $a,b$ 之间（不论 $a,b$ 大小关系）。利用差值符号：$a\le x\le b$ 等价于 $(x-a)$ 和 $(x-b)$ 异号或为零。在不使用减法和比较符的限制下，通过算术右移提取符号位，利用按位运算和逻辑非实现。

**P13 mul5Sat**

计算 $5x$ 并做饱和处理。$5x=(x\ll 2)+x$。通过符号位判断是否溢出：若正数加后变负数，或负数加后变正数，则发生溢出。发生正溢出返回 `0x7FFFFFFF`，负溢出返回 `0x80000000`，否则返回原值。

**P14 classifyAdd3**

对三个数加法结果分类。先计算部分和，提取符号位，计算进位情况。通过判断加数符号和被加数符号是否一致，以及结果符号是否发生翻转，来严谨地检测溢出情况（正溢出、负溢出、正常正数、正常负数、零）。

## 浮点部分 (P15 - P17)

**P15 floatScaleThreeHalves**

浮点数乘以 1.5。将阶码、尾数分离。若阶码全 1（NaN/Inf）直接返回。如果不需进位，可直接修改阶码或尾数；若涉及舍入需要处理进位。利用分支语句分别处理非规格化数、规格化数以及符号位。

**P16 floatRoundEven**

浮点数舍入到整数（四舍六入五成双）。提取阶码，如果阶码 $< 0$ 返回 0（非规格化数等）；如果阶码过大则直接返回原值。在需要截断的区间内，提取尾数及被丢弃的部分，比较被丢弃部分与中点 `0x800000` 的关系。等于中点时，判断保留部分的最低位（即原浮点数阶码处的尾数位）是否为 1。

**P17 float_i2f**

整数转单精度浮点。若 $x = 0$ 返回 0。取绝对值并提取最高有效位（MSB）得到指数。尾数部分需要左移或右移进行对齐。最难处理的是舍入：在对齐后的尾数丢弃低位时，需要判断丢弃位是否大于中点，或等于中点且保留的最低位为 1，从而决定是否向尾数加 1。如果尾数溢出则阶码进位。

## 位运算算法题 (P18 - P19)

**P18 bitCount**

统计二进制中 1 的个数。采用经典的分治算法（Parallel Bit Count）：先每 2 位为一组统计，再每 4 位、8 位、16 位分组累加。利用掩码 `0x55555555`、`0x33333333` 等，通过掩码提取和移位加法完成，效率极高。

**P19 bitReverse**

反转 32 位二进制位。同样采用分治思想：先交换相邻的 1 位，再交换相邻的 2 位，接着是 4 位、8 位、16 位。每次交换通过掩码分离奇偶部分，移位后按位或合并，最终实现完全反转。

# 三、引用与参考资料

在本实验过程中，我独立思考并推导了大部分函数的实现。遇到困难时，参考了以下资料：

- 《深入理解计算机系统》（原书第 3 版），Randal E. Bryant, David R. O'Hallaron 著。（参考了第二章关于补码、位运算以及 IEEE 754 浮点数格式的理论基础）
- 课程提供的实验指南及 `README.md` 中对于各题的要求和边界条件说明。
- 类似分治算法的思想来源于计算机系统结构中常见的 Hacker's Delight（高效位运算技巧）相关理论基础。

注：未直接复制任何网络上的代码，所有代码均由本人根据上述理论推导手写并调试通过。

# 四、实验感受与建议

**实验感受：**这次 Data Lab 实验极具挑战性但也充满成就感。前几道整数题让我深刻理解了补码的特性，比如在防溢出计算和掩码构造时，算术右移的特性起到了关键作用。浮点数部分（尤其是 `floatRoundEven` 和 `float_i2f`）是最折磨人的，为了处理“五成双”的舍入规则和各种边界情况（如非规格化数、NaN），我需要反复使用 `./fshow` 来观察内存中的真实比特位，这极大地提升了我对 IEEE 754 格式的直观认识。最后的 `bitCount` 和 `bitReverse` 让我见识到了分治算法在位运算中的优雅。

**对助教的建议：**感谢助教提供的自动化测试脚本。在调试 `float_i2f` 舍入进位导致阶码溢出的 bug 时，GitHub Actions 的详细日志给了我很大帮助。建议在未来的实验指导中，可以适当增加几个关于浮点数舍入边界的测试用例提示，这能帮助同学们更快地定位自己逻辑上的盲区。